In [ ]:
```
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import scipy.signal as sig
import seaborn as sns
import librosa
import librosa.display
import marimo as mo

PROJECT_ROOT = Path("/mnt/d/Projects/PedalDSP")
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

from pedal_model.eval.results import ZooResults
from pedal_model.signals.manifest import Manifest

RESULTS_PATH = PROJECT_ROOT / "results" / "data" / "klon_sim" / "klon_sim_v1
zoo = ZooResults.load(RESULTS_PATH)
manifest = Manifest(PROJECT_ROOT / "data" / "signals" / "val_signal_v1.json"

sr = zoo.sr
MODEL_NAMES = list(zoo.models.keys())

# Consistent colour palette across all plots
_cmap = plt.cm.tab20
COLOR_MAP = {name: _cmap(i / max(len(MODEL_NAMES) - 1, 1)) for i, name in er

# Section bounds from manifest
SECTION_BOUNDS = {s.label: (s.start_sample, s.end_sample) for s in manifest.
SECTION_TYPES  = {s.label: s.type for s in manifest.sections}
SECTION_LABELS = [s.label for s in manifest.sections]

# Tone sections (have THD/HP metrics)
TONE_LABELS = [s.label for s in manifest.sections_of_type("stepped_sine_tone

sns.set_theme(style="whitegrid", palette="muted", font_scale=0.9)
print(f"Loaded: {len(MODEL_NAMES)} models, {len(SECTION_LABELS)} sections, s
print(f"Tone sections: {len(TONE_LABELS)}, others: {len(SECTION_LABELS) - le
```

```
Loaded: 11 models, 38 sections, sr=96000 Hz
Tone sections: 24, others: 14
```

1. Signal overview — RMS envelope

In [ ]:
```
def _rms_envelope(x, win_samples):
    n = len(x) // win_samples
    return np.sqrt(np.mean(x[: n * win_samples].reshape(n, win_samples) ** 2

win = int(sr * 0.01)  # 10 ms windows
```

```
t_env = np.arange(len(zoo.dry_val) // win) * win / sr

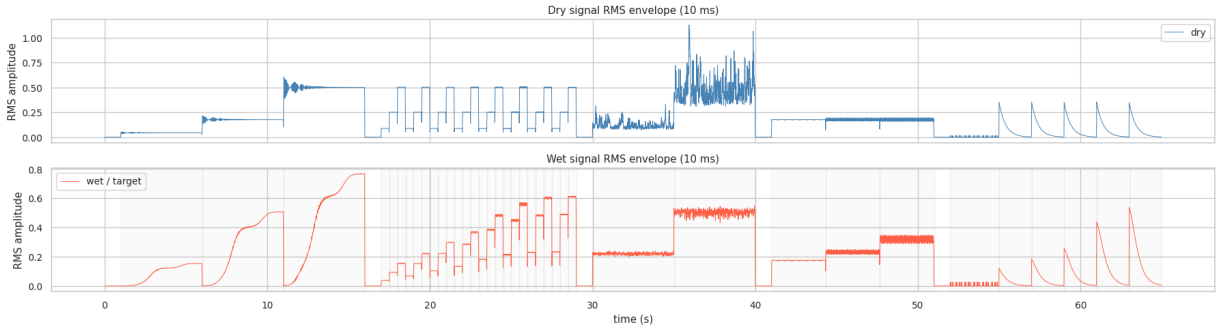
dry_env = _rms_envelope(zoo.dry_val, win)
wet_env = _rms_envelope(zoo.wet_val, win)

fig_ov, axes_ov = plt.subplots(2, 1, figsize=(18, 5), sharex=True)
axes_ov[0].plot(t_env, dry_env, lw=0.7, color="steelblue", label="dry")
axes_ov[0].set_ylabel("RMS amplitude")
axes_ov[0].set_title("Dry signal RMS envelope (10 ms)")
axes_ov[0].legend()

axes_ov[1].plot(t_env, wet_env, lw=0.7, color="tomato", label="wet / target"
axes_ov[1].set_ylabel("RMS amplitude")
axes_ov[1].set_xlabel("time (s)")
axes_ov[1].set_title("Wet signal RMS envelope (10 ms)")
axes_ov[1].legend()

# Shade section regions on wet plot
for sec in manifest.sections:
    t0 = sec.start_sample / sr
    t1 = sec.end_sample   / sr
    axes_ov[1].axvspan(t0, t1, alpha=0.04, color="gray")

plt.tight_layout()
mo.mpl.interactive(fig_ov)
```



2. Summary metrics heatmap (mean across all sections)

In [ ]:

```
_DISPLAY_METRICS = ["ESR", "null_depth_dB", "STFT", "FR_err_dB",
                    "THD_pattern_dist", "HP_sim"]
rows_heat = {}
for _name in MODEL_NAMES:
    _secs = zoo.models[_name].metrics_per_section
    _row = {}
    for _m in _DISPLAY_METRICS:
        _vals = [v[_m] for v in _secs.values() if _m in v and np.isfinite(v[
        _row[_m] = float(np.mean(_vals)) if _vals else float("nan")
    rows_heat[_name] = _row

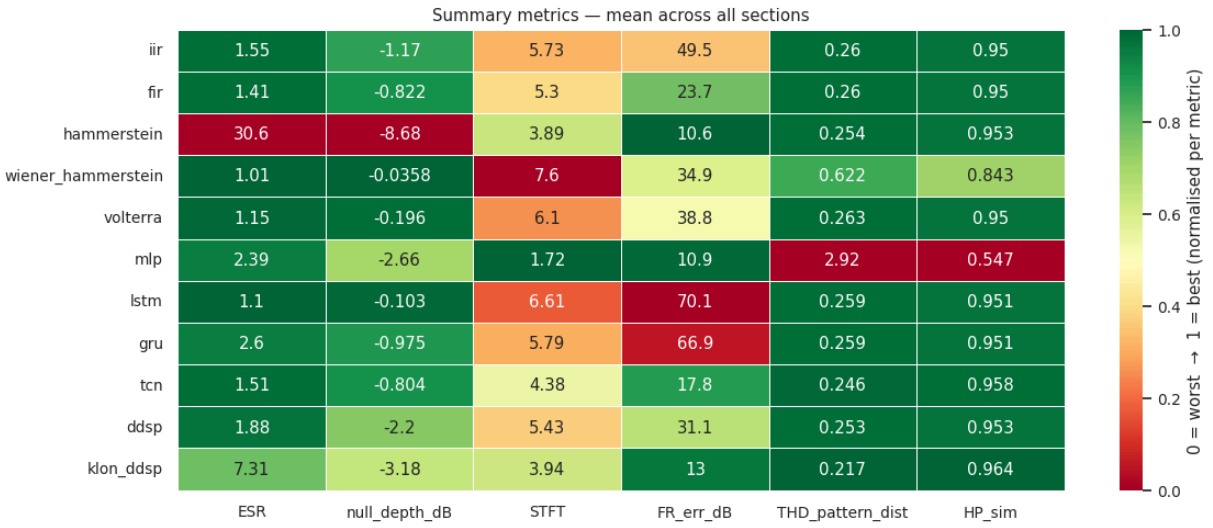
df_heat = pd.DataFrame(rows_heat).T[_DISPLAY_METRICS]

# Per-column normalise to [0,1] for colour; NaN → 0.5
df_norm = df_heat.copy()
```

```
for _col in df_norm.columns:
    _mn, _mx = df_norm[_col].min(), df_norm[_col].max()
    df_norm[_col] = (df_norm[_col] - _mn) / max(_mx - _mn, 1e-9)
df_norm = df_norm.fillna(0.5)

# For ESR/STFT/FR_err/THD_pattern_dist lower is better → invert colour
_invert = {"ESR", "STFT", "FR_err_dB", "THD_pattern_dist"}
for _col in _invert:
    if _col in df_norm.columns:
        df_norm[_col] = 1.0 - df_norm[_col]

fig_heat, ax_heat = plt.subplots(figsize=(12, 5))
sns.heatmap(
    df_norm,
    ax=ax_heat,
    annot=df_heat.map(lambda v: f"{v:.3g}" if np.isfinite(v) else "—"),
    fmt="",
    cmap="RdYlGn",
    linewidths=0.4,
    vmin=0, vmax=1,
    cbar_kws={"label": "0 = worst  →  1 = best (normalised per metric)"},
)
ax_heat.set_title("Summary metrics — mean across all sections")
ax_heat.set_ylabel("")
plt.tight_layout()
mo.mpl.interactive(fig_heat)
```



3. ESR per section (grouped by section type)

In [ ]:

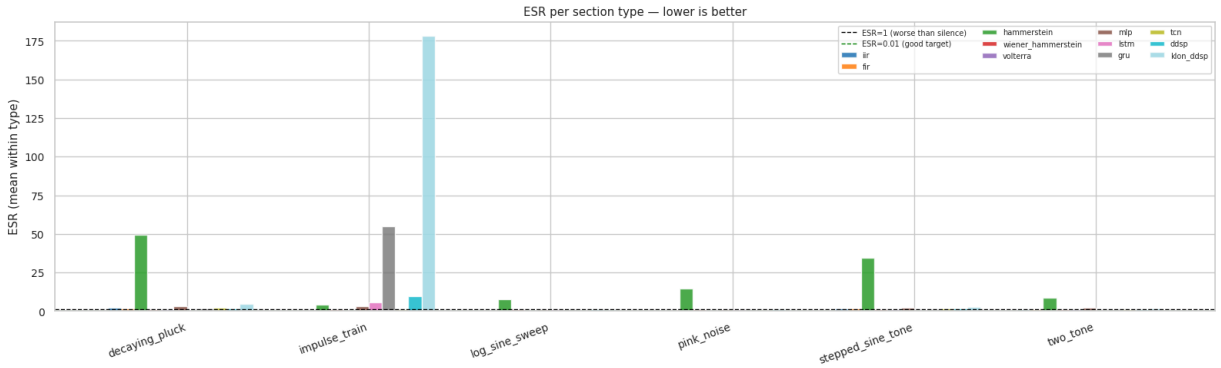
```
_rows_esr = []
for _name in MODEL_NAMES:
    for _sec, _metrics in zoo.models[_name].metrics_per_section.items():
        _rows_esr.append({
            "model": _name,
            "section": _sec,
            "type": SECTION_TYPES.get(_sec, "unknown"),
            "ESR": float(_metrics.get("ESR", float("nan"))),
        })
```

```
df_esr = pd.DataFrame(_rows_esr)

# Mean ESR per model × type
df_esr_grp = df_esr.groupby(["model", "type"])["ESR"].mean().reset_index()
_types = sorted(df_esr_grp["type"].unique())
_x = np.arange(len(_types))
_w = 0.8 / len(MODEL_NAMES)

fig_esr, ax_esr = plt.subplots(figsize=(16, 5))
for _i, _name in enumerate(MODEL_NAMES):
    _sub = df_esr_grp[df_esr_grp["model"] == _name].set_index("type")
    _vals = [float(_sub.loc[t, "ESR"]) if t in _sub.index else float("nan")
    ax_esr.bar(_x + _i * _w - 0.4, _vals, _w, label=_name,
               color=COLOR_MAP[_name], alpha=0.85)

ax_esr.axhline(1.0, color="black", ls="--", lw=1, label="ESR=1 (worse than s
ax_esr.axhline(0.01, color="green", ls="--", lw=1, label="ESR=0.01 (good tar
ax_esr.set_xticks(_x)
ax_esr.set_xticklabels(_types, rotation=20, ha="right")
ax_esr.set_ylabel("ESR (mean within type)")
ax_esr.set_title("ESR per section type — lower is better")
ax_esr.legend(fontsize=7, ncol=4)
ax_esr.set_ylim(bottom=0)
plt.tight_layout()
mo.mpl.interactive(fig_esr)
```



4. Null-test depth per section (dB — higher is better)

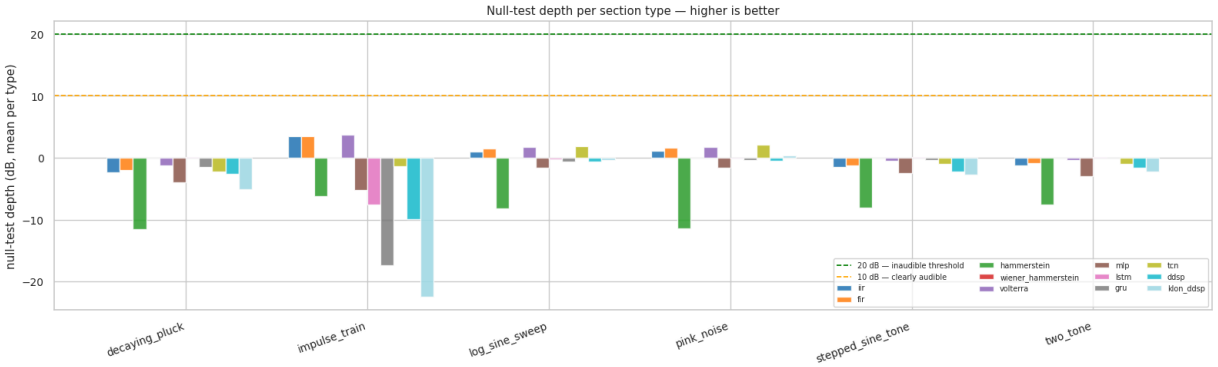
In [ ]:

```
_rows_nd = []
for _name in MODEL_NAMES:
    for _sec, _metrics in zoo.models[_name].metrics_per_section.items():
        _nd = float(_metrics.get("null_depth_dB", float("nan")))
        _nd = min(_nd, 80.0) if np.isfinite(_nd) else float("nan")
        _rows_nd.append({
            "model": _name, "section": _sec,
            "type": SECTION_TYPES.get(_sec, "unknown"),
            "null_depth_dB": _nd,
        })
df_nd = pd.DataFrame(_rows_nd)
df_nd_grp = df_nd.groupby(["model", "type"])["null_depth_dB"].mean().reset_i
```

```
_types = sorted(df_nd_grp["type"].unique())
_x = np.arange(len(_types))
_w = 0.8 / len(MODEL_NAMES)

fig_nd, ax_nd = plt.subplots(figsize=(16, 5))
for _i, _name in enumerate(MODEL_NAMES):
    _sub = df_nd_grp[df_nd_grp["model"] == _name].set_index("type")
    _vals = [float(_sub.loc[t, "null_depth_dB"]) if t in _sub.index else flo
    ax_nd.bar(_x + _i * _w - 0.4, _vals, _w, label=_name,
              color=COLOR_MAP[_name], alpha=0.85)

ax_nd.axhline(20, color="green", ls="--", lw=1.2, label="20 dB — inaudible t
ax_nd.axhline(10, color="orange", ls="--", lw=1.2, label="10 dB — clearly au
ax_nd.set_xticks(_x)
ax_nd.set_xticklabels(_types, rotation=20, ha="right")
ax_nd.set_ylabel("null-test depth (dB, mean per type)")
ax_nd.set_title("Null-test depth per section type — higher is better")
ax_nd.legend(fontsize=7, ncol=4)
plt.tight_layout()
mo.mpl.interactive(fig_nd)
```



5. Waveform overlay

In [ ]:
```
wf_section = mo.ui.dropdown(
    SECTION_LABELS, value=SECTION_LABELS[3],  # first tone section
    label="Section",
)
wf_models = mo.ui.multiselect(
    MODEL_NAMES, value=MODEL_NAMES[:4],
    label="Models to overlay",
)
wf_window_ms = mo.ui.slider(10, 200, value=40, step=5, label="Window (ms)")
wf_offset_ms = mo.ui.slider(0, 500, value=0, step=5, label="Offset into sect
mo.hstack([wf_section, wf_models, wf_window_ms, wf_offset_ms])
```

Section val_stepped_tone_f147_l-18 Models to overlay 4 selected Window (ms) Offset into section (ms)

In [ ]:
```
_sec = wf_section.value
_start0, _end0 = SECTION_BOUNDS[_sec]
_off   = int(wf_offset_ms.value * sr / 1000)
_win   = int(wf_window_ms.value  * sr / 1000)
_start = min(_start0 + _off, _end0 - _win)
_end   = min(_start + _win, _end0)
```

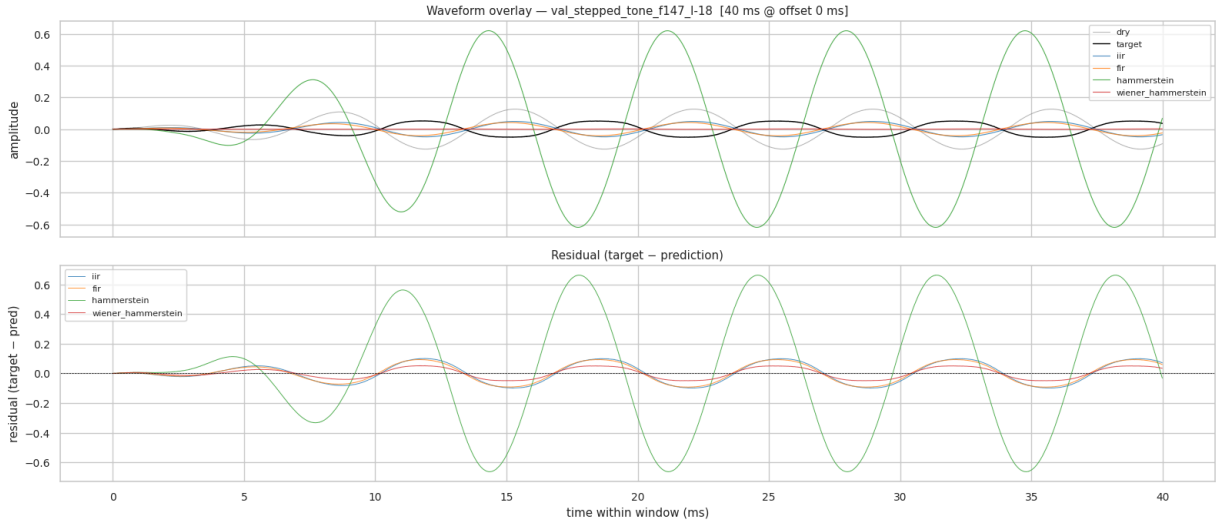
```
dry_w  = zoo.dry_val[_start:_end]
wet_w  = zoo.wet_val[_start:_end]
t_ms   = 1000.0 * np.arange(len(dry_w)) / sr

fig_wf, (ax_wf, ax_res) = plt.subplots(2, 1, figsize=(16, 7), sharex=True)

ax_wf.plot(t_ms, dry_w, color="gray",  lw=0.7, alpha=0.6, label="dry")
ax_wf.plot(t_ms, wet_w, color="black", lw=1.0, label="target")
for _name in wf_models.value:
    _pred = zoo.models[_name].prediction[_start:_end]
    ax_wf.plot(t_ms, _pred, lw=0.8, alpha=0.8, color=COLOR_MAP[_name], label
ax_wf.set_ylabel("amplitude")
ax_wf.set_title(f"Waveform overlay — {_sec}  [{wf_window_ms.value} ms @ offs
ax_wf.legend(fontsize=8)

# Residual (wet - prediction) for each model
for _name in wf_models.value:
    _pred = zoo.models[_name].prediction[_start:_end]
    ax_res.plot(t_ms, wet_w - _pred, lw=0.7, alpha=0.8, color=COLOR_MAP[_nam
ax_res.axhline(0, color="black", lw=0.5, ls="--")
ax_res.set_ylabel("residual (target − pred)")
ax_res.set_xlabel("time within window (ms)")
ax_res.set_title("Residual (target − prediction)")
ax_res.legend(fontsize=8)

plt.tight_layout()
mo.mpl.interactive(fig_wf)
```



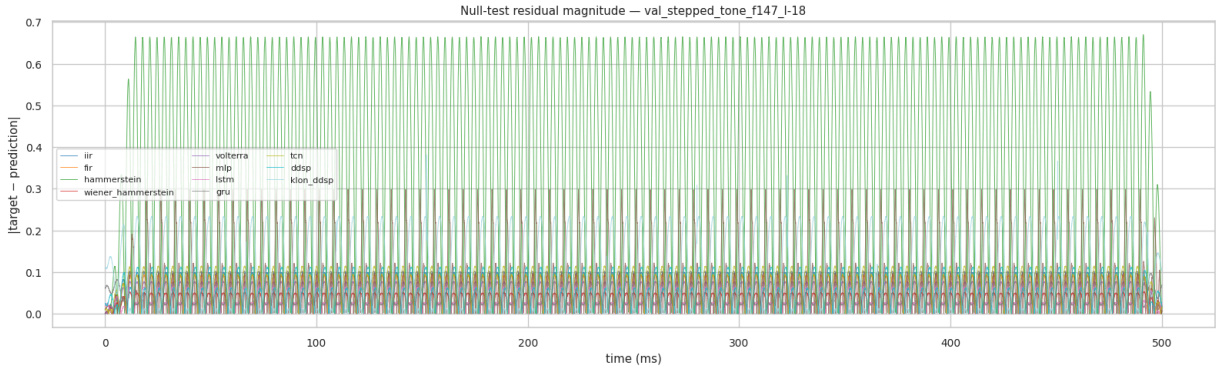
6. Null-test residual waveform — |target − prediction|

In [ ]:
```
nt_section = mo.ui.dropdown(SECTION_LABELS, value=SECTION_LABELS[3], label="
nt_section
```

Section val_stepped_tone_f147_l-18

In [ ]:
```
_sec = nt_section.value
_s, _e = SECTION_BOUNDS[_sec]
wet_nt = zoo.wet_val[_s:_e]
t_nt   = np.arange(len(wet_nt)) / sr * 1000

fig_nt, ax_nt = plt.subplots(figsize=(16, 5))
for _name in MODEL_NAMES:
    _pred = zoo.models[_name].prediction[_s:_e]
    ax_nt.plot(t_nt, np.abs(wet_nt - _pred), lw=0.7, alpha=0.75,
               color=COLOR_MAP[_name], label=_name)
ax_nt.set_xlabel("time (ms)")
ax_nt.set_ylabel("|target − prediction|")
ax_nt.set_title(f"Null-test residual magnitude — {_sec}")
ax_nt.legend(fontsize=8, ncol=3)
plt.tight_layout()
mo.mpl.interactive(fig_nt)
```



7. FFT spectrum comparison

In [ ]:
```
fft_section = mo.ui.dropdown(SECTION_LABELS, value=SECTION_LABELS[0], label=
fft_models  = mo.ui.multiselect(MODEL_NAMES, value=MODEL_NAMES[:5], label="M
mo.hstack([fft_section, fft_models])
```

Section val_sweep_l-24 Models 5 selected

In [ ]:
```
_sec = fft_section.value
_s, _e = SECTION_BOUNDS[_sec]

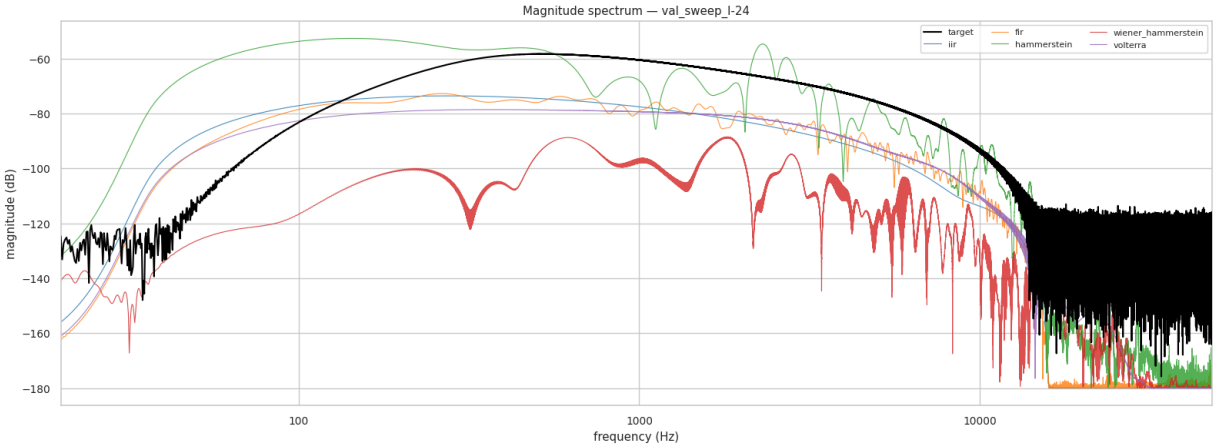
def _mag_db(x):
    N = len(x)
    w = np.hanning(N)
    X = np.fft.rfft(x * w)
    mag = np.abs(X) / (N / 2)
    return np.fft.rfftfreq(N, 1.0 / sr), 20 * np.log10(mag + 1e-9)

fig_fft, ax_fft = plt.subplots(figsize=(16, 6))
_wet_chunk = zoo.wet_val[_s:_e]
_f, _db = _mag_db(_wet_chunk)
ax_fft.semilogx(_f[1:], _db[1:], color="black", lw=1.5, label="target", zorc

for _name in fft_models.value:
    _pred = zoo.models[_name].prediction[_s:_e]
```

```
    _f2, _db2 = _mag_db(_pred)
    ax_fft.semilogx(_f2[1:], _db2[1:], lw=0.8, alpha=0.8,
                    color=COLOR_MAP[_name], label=_name)

ax_fft.set_xlim(20, sr // 2)
ax_fft.set_xlabel("frequency (Hz)")
ax_fft.set_ylabel("magnitude (dB)")
ax_fft.set_title(f"Magnitude spectrum — {_sec}")
ax_fft.legend(fontsize=8, ncol=3)
ax_fft.xaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0f}"))
plt.tight_layout()
mo.mpl.interactive(fig_fft)
```



8. Spectrogram grid — target vs top-4 models by ESR

In [ ]:
```
spec_section = mo.ui.dropdown(SECTION_LABELS, value=SECTION_LABELS[0], label
spec_section
```

Section for spectrogram val_sweep_l-24

In [ ]:
```
_sec = spec_section.value
_s, _e = SECTION_BOUNDS[_sec]

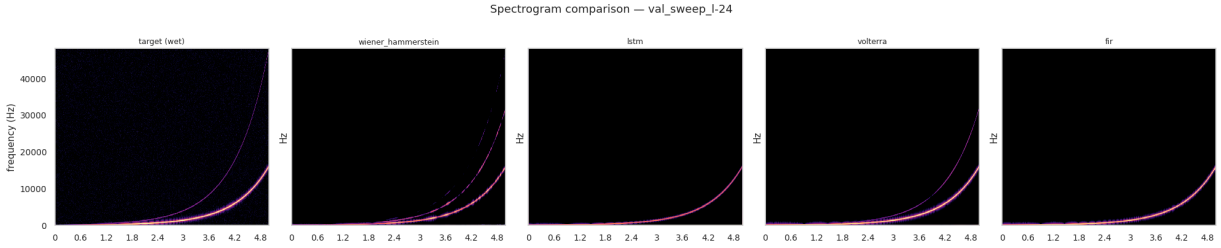
# Rank models by mean ESR ascending
_esr_mean = {}
for _nm in MODEL_NAMES:
    _vals = [v["ESR"] for v in zoo.models[_nm].metrics_per_section.values()
             if "ESR" in v and np.isfinite(v["ESR"])]
    _esr_mean[_nm] = float(np.mean(_vals)) if _vals else 999.0
_top4 = sorted(MODEL_NAMES, key=lambda n: _esr_mean[n])[:4]

_signals = [("target (wet)", zoo.wet_val[_s:_e])]
_signals += [(_nm, zoo.models[_nm].prediction[_s:_e]) for _nm in _top4]

fig_spec, axes_spec = plt.subplots(1, 5, figsize=(20, 4), sharey=True)
for _ax, (_label, _x) in zip(axes_spec, _signals):
    _D = librosa.amplitude_to_db(
        np.abs(librosa.stft(_x.astype(np.float32), n_fft=1024, hop_length=25
        ref=np.max
```

```
    )
    librosa.display.specshow(_D, sr=sr, hop_length=256, x_axis="time", y_axi
                             ax=_ax, vmin=-80)
    _ax.set_title(_label, fontsize=9)
    _ax.set_xlabel("")

axes_spec[0].set_ylabel("frequency (Hz)")
fig_spec.suptitle(f"Spectrogram comparison — {_sec}", y=1.01)
plt.tight_layout()
mo.mpl.interactive(fig_spec)
```



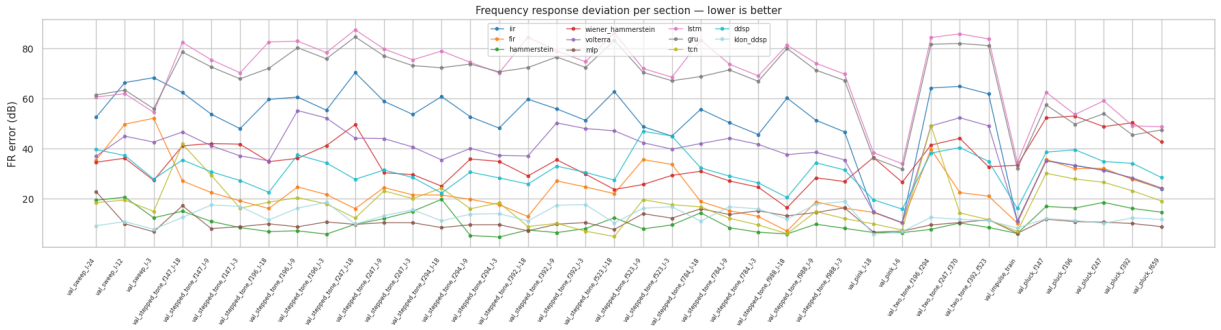
9. Frequency response deviation (FR_err_dB) per section

In [ ]:

```
_n = len(SECTION_LABELS)
_x = np.arange(_n)

fig_fr, ax_fr = plt.subplots(figsize=(18, 5))
for _name in MODEL_NAMES:
    _vals = [
        zoo.models[_name].metrics_per_section.get(_sec, {}).get("FR_err_dB",
        for _sec in SECTION_LABELS
    ]
    ax_fr.plot(_x, _vals, lw=1.0, marker="o", ms=3, alpha=0.8,
               color=COLOR_MAP[_name], label=_name)

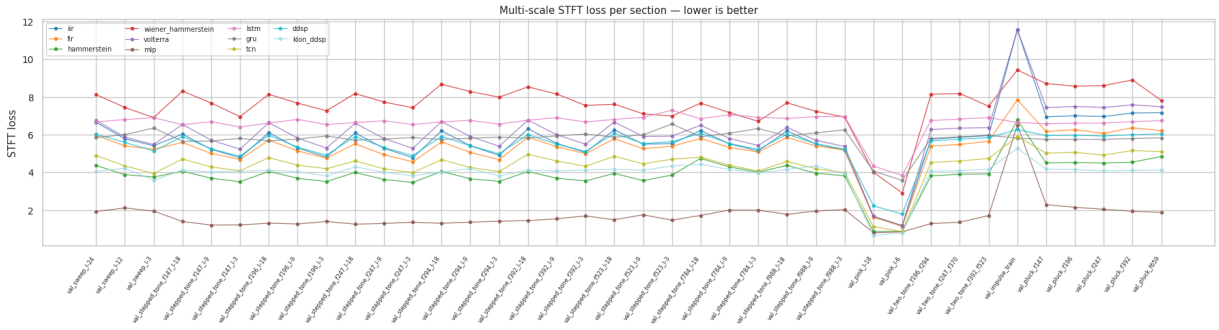
ax_fr.set_xticks(_x)
ax_fr.set_xticklabels(SECTION_LABELS, rotation=55, ha="right", fontsize=6)
ax_fr.set_ylabel("FR error (dB)")
ax_fr.set_title("Frequency response deviation per section — lower is better"
ax_fr.legend(fontsize=7, ncol=4)
plt.tight_layout()
mo.mpl.interactive(fig_fr)
```



10. STFT loss per section

In [ ]:
```
fig_stft, ax_stft = plt.subplots(figsize=(18, 5))
_x = np.arange(len(SECTION_LABELS))
for _name in MODEL_NAMES:
    _vals = [
        zoo.models[_name].metrics_per_section.get(_sec, {}).get("STFT", floa
        for _sec in SECTION_LABELS
    ]
    ax_stft.plot(_x, _vals, lw=0.9, marker="o", ms=3, alpha=0.8,
                 color=COLOR_MAP[_name], label=_name)

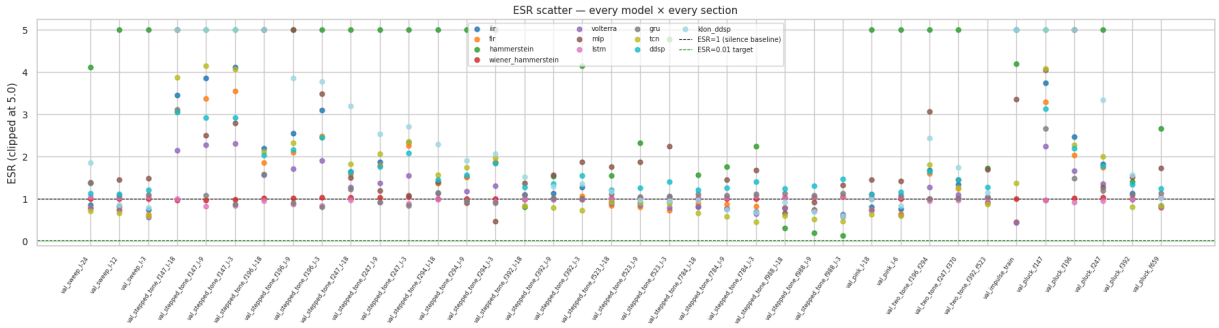
ax_stft.set_xticks(_x)
ax_stft.set_xticklabels(SECTION_LABELS, rotation=55, ha="right", fontsize=6)
ax_stft.set_ylabel("STFT loss")
ax_stft.set_title("Multi-scale STFT loss per section — lower is better")
ax_stft.legend(fontsize=7, ncol=4)
plt.tight_layout()
mo.mpl.interactive(fig_stft)
```



11. ESR scatter across all sections

In [ ]:
```
fig_sc, ax_sc = plt.subplots(figsize=(18, 5))
_x = np.arange(len(SECTION_LABELS))
for _name in MODEL_NAMES:
    _vals = [
        min(zoo.models[_name].metrics_per_section.get(_sec, {}).get("ESR", f
        for _sec in SECTION_LABELS
    ]
    ax_sc.scatter(_x, _vals, s=25, alpha=0.8, color=COLOR_MAP[_name], label=

ax_sc.axhline(1.0, color="black", ls="--", lw=0.8, label="ESR=1 (silence bas
ax_sc.axhline(0.01, color="green", ls="--", lw=0.8, label="ESR=0.01 target")
ax_sc.set_xticks(_x)
ax_sc.set_xticklabels(SECTION_LABELS, rotation=55, ha="right", fontsize=6)
ax_sc.set_ylabel("ESR (clipped at 5.0)")
ax_sc.set_title("ESR scatter — every model × every section")
ax_sc.legend(fontsize=7, ncol=4)
plt.tight_layout()
mo.mpl.interactive(fig_sc)
```

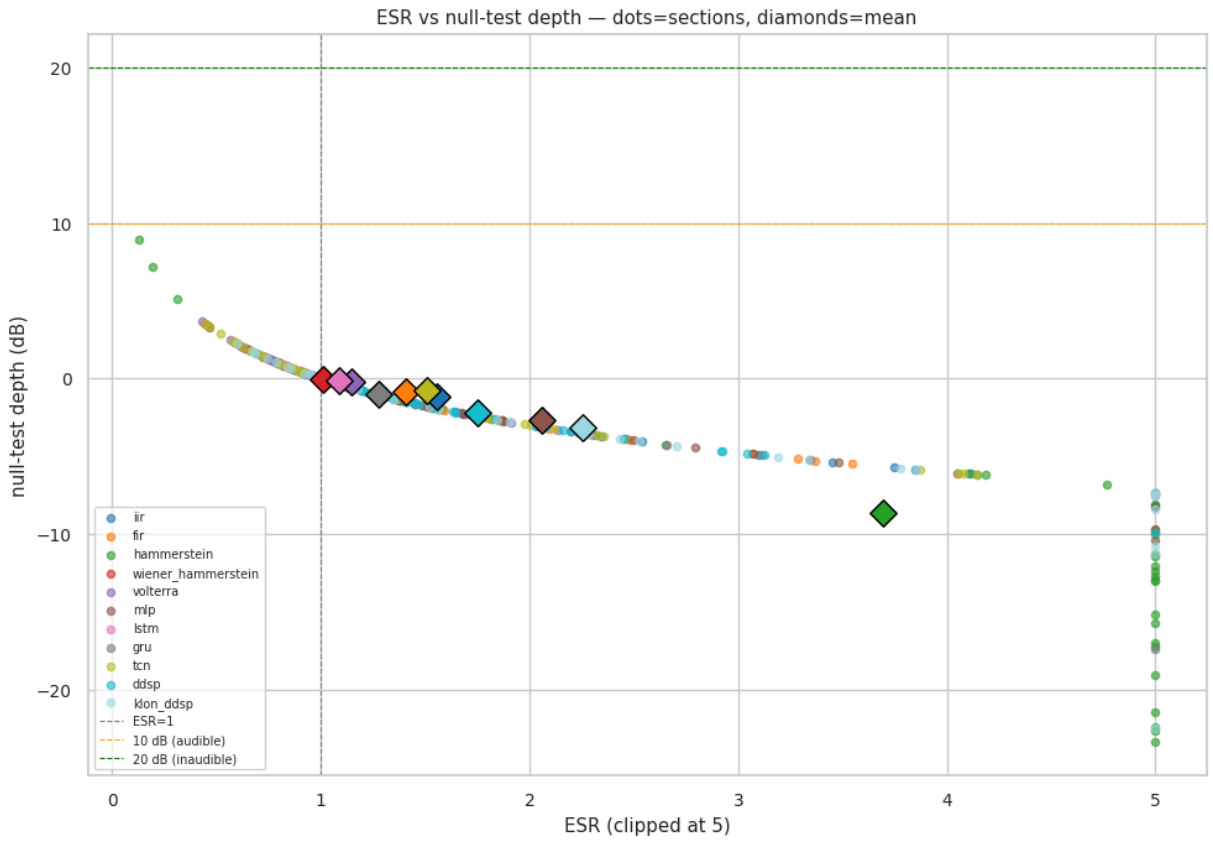


12. ESR vs null-test depth scatter (one dot per model × section)

In [ ]:

```
fig_mv, ax_mv = plt.subplots(figsize=(10, 7))
for _name in MODEL_NAMES:
    _esrs, _nds = [], []
    for _metrics in zoo.models[_name].metrics_per_section.values():
        _e = _metrics.get("ESR", float("nan"))
        _n = _metrics.get("null_depth_dB", float("nan"))
        if np.isfinite(_e) and np.isfinite(_n):
            _esrs.append(min(_e, 5.0))
            _nds.append(np.clip(_n, -30, 80))
    ax_mv.scatter(_esrs, _nds, s=20, alpha=0.6, color=COLOR_MAP[_name], labe
    if _esrs:
        ax_mv.scatter([float(np.mean(_esrs))], [float(np.mean(_nds))],
                      s=120, marker="D", color=COLOR_MAP[_name], edgecolors=

ax_mv.axvline(1.0, color="gray", ls="--", lw=0.8, label="ESR=1")
ax_mv.axhline(10, color="orange", ls="--", lw=0.8, label="10 dB (audible)")
ax_mv.axhline(20, color="green", ls="--", lw=0.8, label="20 dB (inaudible)")
ax_mv.set_xlabel("ESR (clipped at 5)")
ax_mv.set_ylabel("null-test depth (dB)")
ax_mv.set_title("ESR vs null-test depth — dots=sections, diamonds=mean")
ax_mv.legend(fontsize=7)
plt.tight_layout()
mo.mpl.interactive(fig_mv)
```



13. THD analysis — tone sections only

In [ ]:

```
if not TONE_LABELS:
    mo.md("*No tone sections found in results.*")
else:
    # THD_target is the same across models (it's the target signal's THD)
    _first = list(zoo.models.values())[0]
    _thd_target = {
        _sec: _first.metrics_per_section[_sec].get("THD_target", float("nan"
        for _sec in TONE_LABELS if _sec in _first.metrics_per_section
    }

    _n_tone = len(TONE_LABELS)
    _x = np.arange(_n_tone)
    _w = 0.7 / (len(MODEL_NAMES) + 1)

    fig_thd, (ax_thd, ax_thdp) = plt.subplots(2, 1, figsize=(18, 10))

    # THD_pred per model
    for _i, _name in enumerate(MODEL_NAMES):
        _vals = [
            zoo.models[_name].metrics_per_section.get(_sec, {}).get("THD_pre
            for _sec in TONE_LABELS
        ]
        ax_thd.bar(_x + _i * _w - 0.35, _vals, _w, label=_name,
                   color=COLOR_MAP[_name], alpha=0.8)

    # THD_target as step overlay
```

```
    _t_vals = [_thd_target.get(_sec, float("nan")) for _sec in TONE_LABELS]
    ax_thd.plot(_x, _t_vals, color="black", lw=2,
                drawstyle="steps-mid", label="THD_target")
    ax_thd.set_xticks(_x)
    ax_thd.set_xticklabels(TONE_LABELS, rotation=40, ha="right", fontsize=7)
    ax_thd.set_ylabel("THD")
    ax_thd.set_title("THD (predicted vs target) per tone section")
    ax_thd.legend(fontsize=7, ncol=4)

    # THD_pattern_dist per model
    for _i, _name in enumerate(MODEL_NAMES):
        _vals = [
            zoo.models[_name].metrics_per_section.get(_sec, {}).get("THD_pat
            for _sec in TONE_LABELS
        ]
        ax_thdp.bar(_x + _i * _w - 0.35, _vals, _w, label=_name,
                    color=COLOR_MAP[_name], alpha=0.8)
    ax_thdp.set_xticks(_x)
    ax_thdp.set_xticklabels(TONE_LABELS, rotation=40, ha="right", fontsize=7
    ax_thdp.set_ylabel("THD pattern distance")
    ax_thdp.set_title("Harmonic pattern distance per tone section — lower is
    ax_thdp.legend(fontsize=7, ncol=4)

    plt.tight_layout()
    mo.mpl.interactive(fig_thd)
```

14. Harmonic profile similarity (HP_sim) — tone sections

In [ ]:
```
if not TONE_LABELS:
    mo.md("*No tone sections.*")
else:
    _n_tone = len(TONE_LABELS)
    _x = np.arange(_n_tone)
    _w = 0.7 / len(MODEL_NAMES)

    fig_hp, ax_hp = plt.subplots(figsize=(18, 5))
    for _i, _name in enumerate(MODEL_NAMES):
        _vals = [
            zoo.models[_name].metrics_per_section.get(_sec, {}).get("HP_sim"
            for _sec in TONE_LABELS
        ]
        ax_hp.bar(_x + _i * _w - 0.35, _vals, _w, label=_name,
                  color=COLOR_MAP[_name], alpha=0.8)

    ax_hp.axhline(1.0, color="green", ls="--", lw=1, label="1.0 = perfect ma
    ax_hp.set_xticks(_x)
    ax_hp.set_xticklabels(TONE_LABELS, rotation=40, ha="right", fontsize=7)
    ax_hp.set_ylabel("HP similarity (cosine)")
    ax_hp.set_title("Harmonic profile similarity per tone section — higher i
    ax_hp.legend(fontsize=7, ncol=4)
    plt.tight_layout()
    mo.mpl.interactive(fig_hp)
```

15. KlonDDSP: circuit vs ML correction fraction

In [ ]:
```
_r = zoo.models.get("klon_ddsp")
if _r is None:
    mo.md("*klon_ddsp not in results.*")
elif _r.ddsp_analysis is None:
    mo.md(
        """**DDSP fraction analysis not yet computed.**

To populate this, re-run with `analyse_ddsp_fraction` after training:

```python
from pedal_model.models.neural.ddsp_klon import KlonDDSP
# load checkpoint, then:
analysis = model.analyse_ddsp_fraction(zoo.dry_val, zoo.wet_val)
zoo.models['klon_ddsp'].ddsp_analysis = analysis
zoo.save(RESULTS_PATH)
```
"""
)
else:
    _a = _r.ddsp_analysis
    mo.md(
        f"""
Metric	Value
DDSP stage ESR	`{_a['ddsp_esr']:.4f}`
Full model ESR	`{_a['full_esr']:.4f}`
Correction RMS ratio	`{_a['correction_rms_ratio']:.3f}`
DDSP fraction	**{_a['ddsp_fraction']:.1%}**

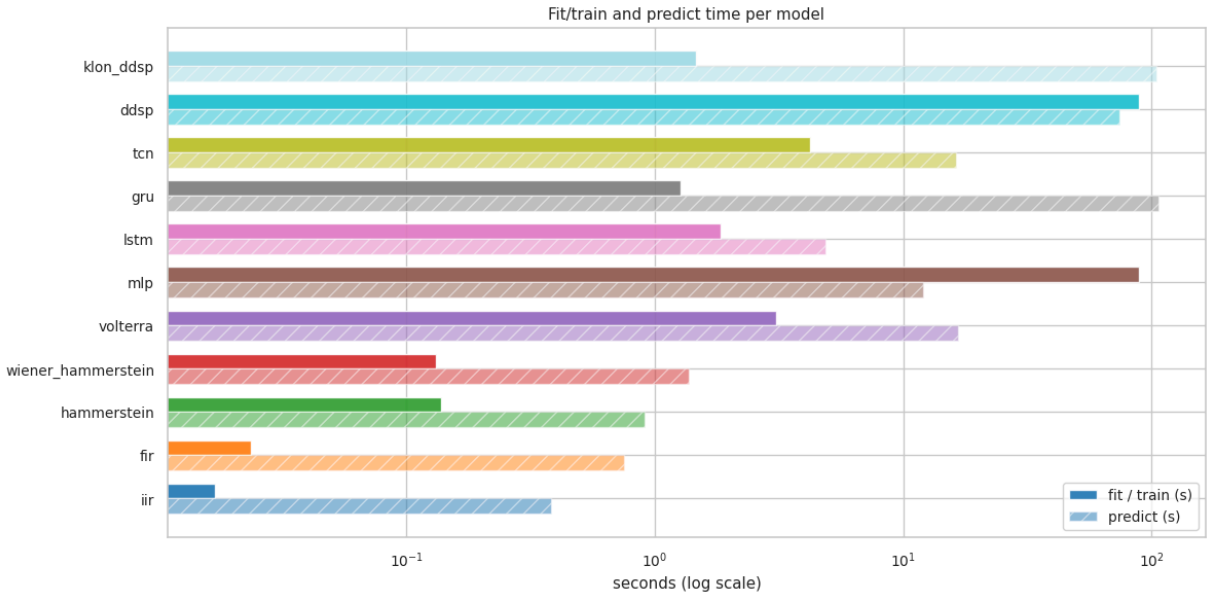
A DDSP fraction near 100 % means the Klon circuit topology accounts for almo
all the pedal response; near 0 % means the GRU is doing all the work.
"""
)
```

16. Timing — fit/train time and predict time

In [ ]:
```
_fit_s   = [zoo.models[n].fit_time_s     for n in MODEL_NAMES]
_pred_s  = [zoo.models[n].predict_time_s for n in MODEL_NAMES]
_y = np.arange(len(MODEL_NAMES))
_h = 0.35

fig_tim, ax_tim = plt.subplots(figsize=(12, 6))
ax_tim.barh(_y + _h / 2, _fit_s,  _h, label="fit / train (s)",
            color=[COLOR_MAP[n] for n in MODEL_NAMES], alpha=0.9)
ax_tim.barh(_y - _h / 2, _pred_s, _h, label="predict (s)",
            color=[COLOR_MAP[n] for n in MODEL_NAMES], alpha=0.5, hatch="//"
ax_tim.set_yticks(_y)
ax_tim.set_yticklabels(MODEL_NAMES)
ax_tim.set_xscale("log")
ax_tim.set_xlabel("seconds (log scale)")
```

```
ax_tim.set_title("Fit/train and predict time per model")
ax_tim.legend()
plt.tight_layout()
mo.mpl.interactive(fig_tim)
```



17. Raw per-section metrics table

In [ ]:
```
raw_model = mo.ui.dropdown(MODEL_NAMES, value=MODEL_NAMES[0], label="Model")
raw_model
```

Model iir

In [ ]:
```
_rows = []
for _sec, _m in zoo.models[raw_model.value].metrics_per_section.items():
    _row = {"section": _sec}
    _row.update({k: round(v, 4) if np.isfinite(v) else "∞" for k, v in _m.it
    _rows.append(_row)
_df = pd.DataFrame(_rows).set_index("section")
mo.ui.table(_df.reset_index())
```

	_marimo_row_id	section	ESR	null_depth_dB	STFT	FR_e
0	0	val_sweep_l-24	0.8474	0.7192	6.6567	52
1	1	val_sweep_l-12	0.8229	0.8466	5.7788	66
2	2	val_sweep_l-3	0.7486	1.2574	5.4147	68
3	3	val_stepped_tone_f147_l-18	3.4532	-5.3822	6.0263	62
4	4	val_stepped_tone_f147_l-9	3.8470	-5.8512	5.2191	53
...	...	...	...	...	...	
33	33	val_pluck_f147	3.7445	-5.7339	6.9423	35
34	34	val_pluck_f196	2.4620	-3.9129	7.0013	33
35	35	val_pluck_f247	1.8160	-2.5911	6.9577	31
36	36	val_pluck_f392	1.1253	-0.5128	7.1484	28
37	37	val_pluck_f659	0.8334	0.7917	7.1648	23

38 rows × 10 columns

18. Aggregated summary table (mean across sections)

In [ ]:
```
_df_summary = zoo.to_dataframe()
mo.ui.table(_df_summary.round(4).reset_index())
```

	_marimo_row_id	model	ESR	null_depth_dB	STFT	FR_err_
0	0	iir	1.5525	-1.1657	5.7326	49.49
1	1	fir	1.4091	-0.8216	5.2975	23.73
2	2	hammerstein	30.5682	-8.6754	3.8879	10.64
3	3	wiener_hammerstein	1.0085	-0.0358	7.6007	34.91
4	4	volterra	1.1463	-0.1964	6.0988	38.78
...	...	...	...	...	...	
6	6	lstm	1.1042	-0.1033	6.6133	70.05
7	7	gru	2.5995	-0.9746	5.7939	66.91
8	8	tcn	1.5079	-0.8037	4.3768	17.75
9	9	ddsp	1.8782	-2.1990	5.4278	31.10
10	10	klon_ddsp	7.3120	-3.1845	3.9425	12.95

11 rows × 12 columns